АЙСМАРТ

Как искусственный интеллект решает задачи - данные, алгоритмы и модели

Занятие 2. Как искусственный интеллект решает задачи - данные, алгоритмы и модели

Содержание

- Из чего состоит простое ИИ-решение
- Шаг 1. Постановка задачи
- Шаг 2. Входные данные
- Шаг 3. Алгоритм обработки
- Шаг 4. Модель
- Чем алгоритм отличается от модели
- Шаг 5. Результат
- Шаг 6. Проверка качества
- Почему ИИ может ошибаться
- Где в этой схеме Python
- Почему Python подходит для ИИ
- ИИ-решение на примере образовательной платформы
- Что важно запомнить
- Самопроверка

Когда мы пользуемся искусственным интеллектом, результат часто выглядит почти волшебно. Чат-бот отвечает на вопрос. Навигатор строит маршрут. Видеосервис предлагает ролик, который действительно хочется посмотреть. Образовательная платформа подбирает задание по теме, где у ученика есть ошибки.

При этом пользователь обычно видит только начало и конец процесса: свой запрос и готовый ответ. Данные, вычисления и проверки остаются внутри системы, поэтому может показаться, что результат возник сам собой.

Но искусственный интеллект не «угадывает» ответ из воздуха. Он работает по понятной схеме: получает данные, обрабатывает их, использует алгоритмы и модели, выдаёт результат, а затем этот результат можно проверить.

Искусственный интеллект можно понимать как технологию, которая помогает решать задачи на основе данных и алгоритмов. Важно, что ИИ создаётся для конкретной цели: одна система распознаёт изображения, другая рекомендует материалы, а третья прогнозирует числовой результат.

Например:

- видеосервис использует данные о просмотрах, чтобы рекомендовать новые ролики;
- навигатор использует данные о дорогах, пробках и скорости движения, чтобы построить маршрут;
- образовательная платформа использует данные об ошибках ученика, чтобы предложить тему для повторения;
- почтовый сервис использует данные о письме, чтобы определить, похоже ли оно на спам;
- интернет-магазин использует данные о покупках и просмотрах, чтобы предложить товары.

Во всех этих случаях ИИ получает информацию, анализирует её и выдаёт результат. Поэтому важно понимать не только то, что ИИ умеет делать, но и из каких шагов складывается ИИ-решение.

Обратите внимание

Наличие программы ещё не означает, что в ней используется искусственный интеллект. Обычный калькулятор тоже получает данные и выдаёт результат, но действует по точно заданным правилам. Для ИИ особенно характерна работа с данными и моделями, которые находят закономерности, строят прогнозы или выбирают подходящую категорию.

Из чего состоит простое ИИ-решение

Чтобы разобрать работу ИИ-системы, полезно разделить её на отдельные этапы. Так можно увидеть, где появляются данные, на каком шаге принимается решение и как обнаружить ошибку.

Простое ИИ-решение можно представить как цепочку:

постановка задачи ☒ входные данные ☒ алгоритм обработки ☒ модель ☒ результат ☒ проверка качества

Эта схема помогает понять, что ИИ — это не одна кнопка и не один файл с программой. Это целый процесс.

Сначала нужно понять, какую задачу мы решаем. Потом определить, какие данные у нас есть. Затем обработать эти данные. После этого использовать модель или алгоритм, получить результат и проверить, насколько он хороший.

Шаги связаны между собой. Например, постановка задачи определяет, какие данные нужно собрать, а тип результата подсказывает, как проверять качество. Если на одном из ранних шагов допущена ошибка, она может повлиять на всю последующую цепочку.

Разберём каждый шаг отдельно.

Шаг 1. Постановка задачи

Любое ИИ-решение начинается с вопроса:

какую задачу мы хотим решить?

Постановка задачи нужна, чтобы не собирать данные и не создавать модель без понятной цели. Она описывает, какую информацию система получает и какой полезный результат от неё ожидается.

Плохая постановка задачи звучит слишком общо:

Сделать умный сервис для обучения.

Непонятно, что именно должен делать этот сервис: проверять ответы, рекомендовать задания, объяснять темы, оценивать прогресс или прогнозировать результат.

Более понятная постановка задачи:

По результатам тренировочных заданий определить, какую тему ученику нужно повторить.

Здесь уже ясно:

- есть ученик;
- есть данные о его решениях;
- нужно определить слабую тему;
- результатом будет рекомендация.

Другой пример:

По данным о просмотрах рекомендовать пользователю похожие видео.

Здесь тоже понятна задача: система должна использовать данные о поведении пользователя и выбрать подходящие ролики.

Хорошую постановку можно проверить дополнительными вопросами:

- для кого решается задача;
- какие данные доступны системе;
- какой результат нужно получить;
- как будет понятно, что результат полезен.

Чем точнее поставлена задача, тем проще понять, какие данные нужны и какой результат должна выдать система.

Частая ошибка

Не следует начинать с формулировки «нужно использовать ИИ». Искусственный интеллект — это средство, а не цель. Сначала определяется проблема, а затем выбирается подходящий способ её решения. Иногда для задачи достаточно обычного алгоритма с несколькими условиями.

Шаг 2. Входные данные

После постановки задачи нужно понять, какие данные будут использоваться.

Входные данные

— это информация, которую получает система перед обработкой. Их можно сравнить с исходными материалами: система не сможет сделать обоснованный вывод о признаке, о котором ей ничего не сообщили.

Если ИИ помогает ученику готовиться к контрольной, входными данными могут быть:

- количество решённых заданий;
- процент правильных ответов;
- темы, где были ошибки;
- время решения;
- количество попыток;
- сложность заданий;
- результаты прошлых тестов.

Если ИИ рекомендует видео, входными данными могут быть:

- история просмотров;
- время просмотра;
- лайки;
- подписки;
- пропущенные ролики;
- поисковые запросы;
- интересы похожих пользователей.

Если ИИ определяет спам, входными данными могут быть:

- текст письма;
- адрес отправителя;
- наличие ссылок;
- вложения;
- частота подозрительных слов;
- история похожих писем.

Отдельные свойства объекта, которые система использует для анализа, часто называют

признаками

. Например, процент правильных ответов, время решения и количество попыток — это признаки,

описывающие работу ученика.

ИИ не может учесть то, чего нет в данных. Например, если образовательная платформа знает только количество решённых заданий, но не знает, сколько из них решены правильно, она может ошибиться. Один ученик мог решать задания внимательно, а другой — просто нажимать ответы случайно. Количество заданий будет одинаковым, но реальный уровень подготовки — разным.

Однако большое количество данных само по себе не гарантирует хороший результат. Данные должны относиться к задаче, быть достаточно точными и корректно записанными. Например, время в минутах и время в секундах нельзя смешивать в одном столбце без преобразования.

Кроме того, при сборе данных важно учитывать правила конфиденциальности: использовать только необходимые сведения и защищать личную информацию пользователей.

Поэтому качество данных очень важно для ИИ.

Мини-проверка

Если нужно определить тему, в которой ученик чаще всего ошибается, что полезнее: общее количество решённых заданий или количество ошибок по каждой теме? Для этой задачи второй вариант содержит более важную информацию.

Шаг 3. Алгоритм обработки

Алгоритм

— это последовательность действий, которые нужно выполнить для решения задачи.

Алгоритмы есть не только в программировании. В жизни мы тоже часто пользуемся алгоритмами.

Например, алгоритм приготовления чая:

1. Вскипятить воду.
2. Положить чай в кружку.
3. Залить кипятком.
4. Подождать несколько минут.
5. Добавить сахар или лимон по желанию.

Порядок действий здесь имеет значение: если поменять некоторые шаги местами или пропустить их, результат изменится. Так же и программа должна выполнять операции в предусмотренной последовательности.

В ИИ-системах алгоритмы нужны для обработки данных и принятия промежуточных решений. Например, алгоритм может удалить ошибочные записи, вычислить среднее значение, сравнить показатель с границей или передать подготовленные данные модели.

Пример простого алгоритма для образовательной платформы:

1. Получить процент правильных ответов ученика.
2. Если процент ниже 60%, предложить повторить тему.

3. Если процент от 60% до 85%, предложить дополнительные задания.
4. Если процент выше 85%, предложить более сложный уровень.

Здесь важно заранее определить, к какой группе относятся граничные значения. Например, значение 60% не подходит под условие «ниже 60%» и будет проверяться следующим условием.

Такой алгоритм можно записать на Python:

```
correct_percent = 78

if correct_percent < 60:
 recommendation = "Повторить тему с начала"
elif correct_percent < 86:
 recommendation = "Решить ещё несколько заданий"
else:
 recommendation = "Перейти к сложному уровню"

print(recommendation)
```

Переменная

correct_percent

хранит входное значение, а условные конструкции выбирают одну из трёх рекомендаций. При значении 78 первое условие ложно, второе истинно, поэтому программа выбирает рекомендацию «Решить ещё несколько заданий».

Эта программа ещё не является сложной ИИ-моделью. Но она уже показывает важную идею: программа получает данные, обрабатывает их по правилам и выдаёт результат.

Шаг 4. Модель

Модель

— это система, которая использует данные, чтобы находить закономерности и делать выводы.

Модель можно представить как настроенный механизм преобразования входных данных в результат. Настройка выполняется на примерах: модель сопоставляет известные входные данные с известными ответами и изменяет свои внутренние параметры, чтобы уменьшить ошибки.

Простое правило можно написать вручную:

Если процент правильных ответов ниже 60%, предложить повторить тему.

Но такое правило не всегда достаточно гибкое. Оно не учитывает, какие именно задания решал ученик, насколько они были сложными, сколько времени он тратил, повторялись ли ошибки, улучшался ли результат со временем.

Модель может работать сложнее. Она может обучаться на данных.

Обучение модели

означает поиск таких внутренних настроек, при которых модель лучше справляется с примерами, для которых правильный результат уже известен.

Например, у нас есть данные о многих учениках:

Решено заданий	Процент правильных ответов	Время подготовки	Итоговый балл
10	60%	2 часа	55
20	75%	4 часа	72
30	85%	6 часов	88

В этой таблице первые три столбца можно использовать как входные признаки, а последний столбец — как известный результат, который модель учится прогнозировать.

Модель анализирует такие примеры и пытается найти закономерности. После этого она может получить данные нового ученика и сделать прогноз.

Например, если новый ученик решил 25 заданий, имеет 80% правильных ответов и готовился 5 часов, модель может предположить его итоговый результат.

При этом по трём строкам таблицы нельзя построить надёжную модель для всех учеников: это лишь упрощённый пример структуры данных. Для обучения обычно нужны разнообразные примеры, отражающие разные возможные ситуации.

Важно: модель не знает будущее точно. Она делает прогноз на основе данных, которые видела раньше. Прогноз является оценкой, а не гарантией.

Важно

Если модель хорошо запомнила примеры для обучения, это ещё не доказывает, что она будет правильно работать с новыми данными. Поэтому качество проверяют на примерах, которые не использовались при настройке модели.

Чем алгоритм отличается от модели

Алгоритм и модель связаны, но это не одно и то же.

Алгоритм

— это заранее заданный порядок действий.

Модель

— это система, которая находит закономерности в данных и использует их для результата.

Простой пример:

- алгоритм — это инструкция;
- модель — это система, которая научилась на примерах.

Если программа работает только по правилу «если меньше 60%, повторить тему», это алгоритм с условием. Границу 60% заранее выбрал разработчик.

Если система изучила много примеров учеников и сама научилась прогнозировать результат, это уже работа модели. Разработчик задаёт способ обучения и подготавливает данные, но конкретные закономерности модель извлекает из примеров.

В реальных ИИ-системах алгоритмы и модели часто используются вместе. Алгоритмы помогают подготовить данные, запустить модель и обработать результат. Модель помогает сделать прогноз, классификацию или рекомендацию.

Например, программа может сначала алгоритмически проверить, заполнены ли все поля, затем передать признаки модели, а после прогноза применить правило: если уверенность системы слишком низкая, отправить результат на дополнительную проверку.

Частая ошибка

Модель не «пишет себе алгоритм» в обычном смысле. Во время обучения изменяются её внутренние параметры. Сам процесс обучения, подготовки данных и использования результата всё равно организован алгоритмами и программным кодом.

Шаг 5. Результат

Результат ИИ-системы зависит от задачи.

ИИ может выдавать разные типы результатов. Тип результата следует определить ещё при постановке задачи: число, категория и рекомендация проверяются по-разному.

Прогноз

Прогноз — это числовое предположение.

Например:

- ученик может получить 78 баллов;
- поездка займёт 35 минут;
- завтра купят 120 товаров;
- пользователь с вероятностью 70% заинтересуется курсом.

Число в прогнозе не следует воспринимать как точный факт. Например, прогноз времени поездки может измениться из-за новых пробок, а предполагаемый балл — из-за сложности конкретного теста.

Классификация

Классификация — это выбор класса или категории.

Например:

- письмо: спам или не спам;
- изображение: кот, собака или машина;
- отзыв: положительный или отрицательный;
- операция: обычная или подозрительная.

Классов может быть два или больше. Важно заранее определить их так, чтобы полученный ответ можно было однозначно использовать и проверить.

Рекомендация

Рекомендация — это предложение подходящего варианта.

Например:

- посмотреть видео;
- послушать песню;
- купить товар;
- повторить тему;
- решить задание;
- пройти курс.

Рекомендация не обязательно является единственно правильным вариантом. Система может составить список из нескольких подходящих материалов, а пользователь выберет один из них.

Решение

Иногда ИИ-система не просто показывает информацию, а выбирает действие.

Например:

- предложить ученику повторить тему;
- отправить операцию на дополнительную проверку;
- показать товар выше в списке;
- выбрать маршрут;
- изменить уровень сложности задания.

Если действие может заметно повлиять на человека, полезно предусмотреть возможность проверки или изменения автоматического решения.

Главное — результат должен соответствовать задаче. Если задача была «определить спам», результатом должен быть класс. Если задача была «предсказать балл», результатом должно быть число.

Шаг 6. Проверка качества

После получения результата нужно понять, насколько он хороший. Без проверки нельзя отличить полезную модель от системы, которая лишь иногда выдаёт правдоподобные ответы.

ИИ может ошибаться. Поэтому результат нужно проверять, сравнивая его с правильным или фактически полученным ответом.

Пример с прогнозом:

Модель предсказала ученику 80 баллов, а он получил 70.

Ошибка составила 10 баллов. Чтобы оценить модель целиком, одного случая недостаточно: нужно посмотреть на ошибки во многих примерах.

Пример с классификацией:

Модель решила, что письмо не является спамом, но оно оказалось спамом.

Это ошибка классификации. Возможна и обратная ошибка: полезное письмо может быть ошибочно помещено в спам.

Проверка качества помогает понять:

- можно ли доверять модели;
- насколько часто она ошибается;
- какие ошибки она делает;
- нужно ли улучшать данные;
- нужно ли менять алгоритм или модель.

Качество оценивают по заранее выбранному критерию. Для прогноза можно измерять величину отклонения от фактического значения, а для классификации — долю правильно определённых примеров. При этом одного показателя иногда недостаточно: важно учитывать, какие именно ошибки наиболее критичны.

Если модель часто ошибается, её нельзя просто использовать без проверки. Важные ИИ-системы обязательно оценивают по качеству.

Например, ошибка в рекомендации видео обычно не очень опасна. Пользователь просто не посмотрит ролик. Но ошибка в банковской безопасности, медицине или образовательной траектории может быть серьёзнее.

Обратите внимание

Требуемое качество зависит от задачи. Одинаковая доля ошибок может быть допустимой для развлекательных рекомендаций и недопустимой для системы, от которой зависит важное решение.

Почему ИИ может ошибаться

ИИ может ошибаться по разным причинам. Ошибка не всегда означает, что программа сломана: иногда модель встретила редкую ситуацию, которой не было среди примеров для обучения.

Данных мало

Если система видела мало примеров, ей сложно найти надёжную закономерность.

Например, если музыкальное приложение знает только одну песню, которую вы послушали, оно ещё не может хорошо понять ваши вкусы. Одна песня может быть случайным выбором и не отражать постоянные интересы.

Данные некачественные

В данных могут быть ошибки, пропуски или неточности.

Например, ученик решил 20 заданий, но система случайно записала 200. Модель может сделать неправильный вывод. Перед использованием данные проверяют: ищут невозможные значения, пропуски и записи в неправильном формате.

Не хватает важных признаков

Модель может учитывать количество решённых заданий, но не учитывать процент правильных ответов. Тогда прогноз будет неполным.

При этом добавлять все доступные сведения без разбора тоже неверно. Признаки должны быть связаны с задачей и допустимы для использования.

Ситуация изменилась

Модель обучалась на одних данных, а используется в новых условиях.

Например, раньше пользователи часто покупали товар из-за низкой цены, а потом появилась дорогая доставка. Старые закономерности могут работать хуже. Поэтому качество модели полезно проверять не только перед запуском, но и во время её дальнейшего использования.

Задача слишком сложная

Некоторые задачи невозможно решить идеально. Реальный мир сложен, и даже хорошая модель может ошибаться.

Например, итоговый результат ученика зависит не только от истории заданий, но и от множества обстоятельств, которых может не быть в данных. Модель способна оценить вероятный результат, но не гарантировать его.

Важно

Уверенно сформулированный ответ ИИ не обязательно является правильным. Результат следует проверять по данным, источникам или фактическому исходу — особенно если ошибка может привести к серьёзным последствиям.

Профессии для ИИ-разработчика

Разобранная схема включает задачи, с которыми работают специалисты в сфере искусственного интеллекта: постановку задачи, анализ данных, обучение модели, программирование и проверку качества.

Профессии в сфере искусственного интеллекта называются по-разному в зависимости от конкретных задач — от создания алгоритмов до интеграции нейросетей в продукты. Главные из них включают:

- ML-инженер (Machine Learning Engineer): специалист, который обучает модели, настраивает алгоритмы машинного обучения и внедряет их в готовые продукты.
- Дата-сайентист (Data Scientist): специалист, который анализирует массивы данных, выявляет закономерности и создаёт математические прототипы ИИ-моделей.
- AI-инженер (инженер по искусственному интеллекту): разработчик, который проектирует ИИ-решения, дообучает большие языковые модели (LLM) и создаёт генеративные приложения (GenAI).
- NLP-инженер / CV-инженер: специалисты по отдельным направлениям. NLP-инженеры разрабатывают системы для обработки человеческого языка, а CV-инженеры работают с компьютерным зрением — например, с распознаванием объектов на изображениях.

В одном проекте эти специалисты могут работать вместе. Например, дата-сайентист исследует данные и проверяет идею модели, а ML-инженер превращает прототип в устойчивую часть работающего приложения.

Где в этой схеме Python

Python — один из самых популярных языков программирования для работы с данными и искусственным интеллектом.

В этом курсе Python нужен не просто для того, чтобы «учить программирование». Он нужен как инструмент для создания ИИ-решений.

С помощью Python можно:

- записывать алгоритмы;
- хранить данные;
- обрабатывать таблицы;
- считать показатели;
- строить графики;
- запускать модели;
- проверять качество результата.

Таким образом, один язык можно применять на разных этапах: от первичной проверки данных до получения прогноза и сравнения его с правильным ответом.

Например, можно сохранить данные об ученике:

```
student = {
"tasks_solved": 25,
"correct_percent": 78,
"mistakes_topic": "Функции"
}

print(student)
```

Словарь

student

объединяет несколько связанных признаков одного ученика. По ключам можно обращаться к отдельным значениям, например к проценту правильных ответов или теме ошибок.

Можно посчитать средний балл группы:

```
scores = [55, 72, 88, 64, 91]

average_score = sum(scores) / len(scores)

print("Средний балл:", average_score)
```

Здесь функция

sum()

вычисляет сумму баллов, а

len()

— количество элементов. Для указанных данных средний балл равен 74.

Можно записать простое правило для рекомендации:

```
correct_percent = 52
mistakes = 8

if correct_percent < 60 and mistakes > 5:
 print("Рекомендуем повторить тему и разобрать ошибки")
elif correct_percent < 80:
 print("Рекомендуем решить дополнительные задания")
else:
 print("Можно перейти к следующей теме")
```

Первое условие проверяет сразу два признака. Оператор

and

означает, что истинными должны быть оба сравнения: процент ниже 60 и количество ошибок больше 5.

Такие примеры показывают связь между Python и ИИ. Программа получает данные, обрабатывает их и выдаёт результат.

Позже вместо простых правил можно использовать модели, которые обучаются на большом количестве примеров. При этом код по-прежнему будет нужен для загрузки данных, запуска обучения и проверки результата.

Почему Python подходит для ИИ

Python часто используют в искусственном интеллекте по нескольким причинам.

Во-первых, у него понятный синтаксис. Команды Python легко читать, особенно на первых этапах обучения. Это позволяет сосредоточиться на данных и логике решения, а не только на записи команд.

Во-вторых, у Python есть много готовых библиотек для работы с данными, графиками и моделями.

Библиотека — это набор готовых инструментов, которые можно использовать в программе. Она не решает задачу автоматически: разработчику всё равно нужно выбрать данные, настроить обработку и проверить результат.

Например:

- для работы с таблицами используют pandas;
- для графиков используют matplotlib;
- для машинного обучения используют scikit-learn;
- для нейронных сетей используют PyTorch или TensorFlow.

Выбор библиотеки зависит от задачи. Для вычисления простого среднего значения достаточно

встроенных возможностей Python, а для обучения модели на таблице может понадобиться специализированная библиотека.

На первых занятиях не нужно знать все эти инструменты. Главное — понять роль Python: он помогает пройти путь от данных к работающему решению.

ИИ-решение на примере образовательной платформы

Рассмотрим всю схему целиком. Такой разбор позволяет проверить, что каждый этап связан с исходной задачей и приводит к измеримому результату.

Постановка задачи

Образовательная платформа хочет определить, какую тему ученику нужно повторить.

Ожидаемый результат формулируется конкретно: название темы и подборка заданий для повторения.

Входные данные

Система собирает данные:

- какие задания решал ученик;
- где ошибался;
- сколько времени тратил;
- сколько попыток делал;
- какие темы уже проходил.

Перед анализом следует проверить, что задания правильно связаны с темами, а результаты и время решения записаны без пропусков и невозможных значений.

Алгоритм обработки

Программа группирует ошибки по темам, считает процент правильных ответов и выделяет сложные места.

Например, если ошибки встречались в темах «Функции» и «Уравнения», программа отдельно подсчитывает показатели для каждой темы. Простое общее количество ошибок не покажет, какую именно тему нужно повторить.

Модель

Модель анализирует данные и определяет, какая тема вызывает наибольшие трудности.

Она может учитывать несколько признаков одновременно: долю ошибок, число попыток, сложность заданий и изменение результата со временем.

Результат

Система рекомендует повторить конкретную тему и предлагает подходящие задания.

Рекомендация должна быть понятной и применимой: одного сообщения «результат низкий» недостаточно, если система не объясняет, какой следующий шаг предлагается.

Проверка качества

Система проверяет, улучшился ли результат ученика после рекомендации.

Для этого можно сравнить выполнение заданий по той же теме до и после повторения. Сравнение должно учитывать сопоставимую сложность заданий, иначе рост или снижение результата может быть связан не с рекомендацией, а с различием самих заданий.

Если ученик стал лучше решать задания, рекомендация могла быть полезной. Однако по одному случаю нельзя оценить всю систему: нужно проверить результаты у разных учеников и на разных темах. Если улучшения нет, следует выяснить причину и доработать данные, алгоритм, модель или способ выдачи рекомендации.

Что важно запомнить

Искусственный интеллект решает задачи по понятной схеме:

постановка задачи ☒ входные данные ☒ алгоритм обработки ☒ модель ☒ результат ☒ проверка качества

ИИ не работает магически. Он использует данные, алгоритмы и модели.

Данные описывают ситуацию. Алгоритмы задают порядок обработки. Модель находит закономерности. Результат показывает ответ системы. Проверка качества помогает понять, насколько этому ответу можно доверять.

Каждый этап влияет на остальные. Нечёткая задача приводит к непонятному результату, ошибки в данных ухудшают выводы модели, а отсутствие проверки не позволяет вовремя заметить проблему.

Python нужен в курсе как инструмент, который помогает работать с данными, записывать алгоритмы, запускать модели и проверять результат.

Если кратко:

ИИ — это путь от задачи и данных к результату. Python помогает пройти этот путь на практике.

Материалы к занятию

- presentation_Как искусственный интеллект решает задачи - данные, алгоритмы и модели.pptx (1.3 МБ)